SQL_TEST_01 · COMMAND REFERENCE · LIVING DOCUMENT

Everything you've typed so far

Every command, meta-command, and function actually used hands-on across this project — grouped by tool, in the order you met them, not alphabetized — plus a project-wide file map, and a running list of Claude's own working commands. Appended at the end of each build, not written once and frozen.

COVERS

Build 1 (agent) through Build 7 (verifier/critic agent) — all complete

ENVIRONMENT

Windows 11 · PowerShell · Docker Desktop · GitHub Actions

DATABASES

pg_local (appdb)

Project Files · Docker · psql · PowerShell · Git · GitHub CLI · psycopg · psycopg.sql · pandas · pandera · pytest · pgvector & embeddings · BeautifulSoup · Python stdlib · Streamlit · Anthropic SDK · Claude's Commands

Project Files

WHAT'S ACTUALLY IN THIS FOLDER

A map of the project root as of the 2026-08-05 repo reorganization — what each file is for, not what commands it contains (that's every section below). Excludes auto-generated folders (`.venv/`, `__pycache__/`, `.git/`).

Reorg, 2026-08-05: a full project audit (prompted by a real CI failure — see the Claude's Commands section for what it found) turned up a flat ~50-file root that had never been reorganized as the project grew across seven builds, plus two stray files and a badly out-of-date `.env.example`. Everything below reflects the result: first-party Python in `src/` (kept flat — the import graph between `agent.py`/`tools.py`/`verify_answer.py`/the eval harnesses rules out further splitting without adding `sys.path` config), SQL migrations in `migrations/`, tests in `tests/` (`pytest.ini` gained `pythonpath = src`), diagrams/briefs/cheat sheet in `docs/` (renamed from `PROJECT_DIAGRAMS/`), and the two vestigial SQL Server lab files in `legacy/`. `db_test_py.py` and an empty VS Code scratch file were deleted outright — same disposition Build 5 gave `api_test.py`.

SQL migrations — migrations/, run in order, numbered for reproducibility

`migrations/001_schema.sql`

Original appdb schema — `customers`/`orders`, the seed data every later build extends.

`migrations/002_seed.sql`

Idempotent seed data for `customers`/`orders`.

`migrations/003_readonly_role.sql`

`appdb_reader` — the read-only role Build 1's agent runs as.

`migrations/004_raw_schema.sql`

Build 2 Phase 1 — `raw.allocations`, all-`TEXT` ingestion target.

`migrations/005_clean_schema.sql`

Build 2 Phase 2 — `clean.allocations`, real types plus the computed `duration_seconds` column.

`migrations/006_agent_scratch_role.sql`

Build 2.5 Phase 1 — `appdb_agent_writer`, the agent's write-scoped role.

`migrations/007_agent_scratch_schema.sql`

Build 2.5 Phase 1 — the `agent_scratch` schema and its grants.

`migrations/008_add_summary_embedding.sql`

Build 3 Phase 1 — adds `clean.allocations.summary_embedding vector(384)`.

`migrations/009_add_summary_embedding_hnsw_index.sql`

Build 3 follow-up — HNSW index on `summary_embedding` for indexed similarity search.

`migrations/010_docs_chunks_schema.sql`

Build 3.5 Phase 1 — the `docs.chunks` table (`vector(384)`) for the documentation RAG corpus.

`migrations/011_metrics_history_schema.sql`

Build 4 follow-up — `observability.metrics_history`, a tracked time series over `metrics_rollup.py`'s runs.

`migrations/012_chat_rounds_schema.sql`

Build 6 Phase 3.5 — `agent_scratch.chat_rounds`, persistent shared chat history surviving a real container restart. Missing from CI's hardcoded migration list from the day it was written until the 2026-08-05 audit caught it — see Claude's Commands.

`migrations/013_allocation_items_schema.sql`

2026-08-06 — `clean.allocation_items`, one row per checked-out item (`item_name`, `category`, `is_accessory`, `is_returned`, `tag`), replacing query-time parsing of `summary`'s delimited text. Populated by `src/build_allocation_items.py`, not SQL alone — see the 2026-08-06 token-burn incident audit for why this exists.

`migrations/014_scratch_table_metadata.sql`

2026-08-10 — `agent_scratch.table_metadata` (`table_name` primary key, `created_at`), closing half of a Build 2.5 deferral ("no cleanup policy for scratch tables"). Age only, not last-use - a deliberate, noted simplification. `appdb_agent_writer` granted `SELECT, INSERT, DELETE` only, deliberately not `UPDATE` - the real code path only ever inserts (on table creation) or deletes (on cleanup); backdating a row to simulate time passing for tests uses the superuser connection instead of loosening this table's real grants.

`migrations/015_demo_token_log.sql`

2026-08-11 — `agent_scratch.demo_token_log` (`id`, `logged_at`, `tokens_used`), a hard daily token-spend cap for the public Streamlit Community Cloud demo (`src/app_demo.py`). INSERT-only for `appdb_agent_writer` - same reasoning as `table_metadata` above: the day's total is a `SUM(...) WHERE logged_at::date = CURRENT_DATE` computed at read time, not a running-total row, so nothing ever needs `UPDATE`. `src/app.py` (local, single-operator) never touches this table at all - the cap exists only for the public link, which has no login and could otherwise run up an unbounded bill.

Python — the agent itself (src/)

`src/agent.py`

The hand-built tool-use loop, wrapped in a callable `ask_agent()` since Build 4 — calls the Anthropic API, dispatches tool calls, enforces `MAX_TOOL_TURNS`, tags every logged call with a `question_id`. Build 7 — every successful answer with real tool evidence now also runs through `verify_answer()` before returning, wrapped in `try/except` so a verifier failure can never block the real answer. 2026-08-06 — tracks per-turn Anthropic token usage (printed to console, logged via `log_final_answer`); `sys.stdout.reconfigure(encoding="utf-8")` added after a real `UnicodeEncodeError` crash (Windows' default `cp1252` console encoding can't represent characters Claude's own output routinely uses, e.g. →). `SYSTEM_PROMPT` went through 5 iterations the same day chasing a 177,090-token/wrong-answer question — the first four tried to teach the model to parse `summary`'s delimited text correctly (aggregate queries, accessory-keyword traps, a mandatory full-list-first rule); each fixed one symptom while introducing another. The prompt actually got *shorter* once `clean.allocation_items` (migration 013) existed to point at instead — full account in the 2026-08-06 token-burn incident audit. 2026-08-07 — the `docs.chunks` guidance's hardcoded `source_file` string fixed (see the deployment-verification audit), then extended twice more same day: "latest/most recent X" questions steered toward `run_sql_query ... ORDER BY chunk_id DESC` rather than raw `search_docs`, after a live test proved semantic similarity has no concept of chronological order; then the SQL-routing rule itself widened from matching specific phrasings ("list every X") to matching the underlying category (files/commands/migrations, regardless of how the question is asked), plus an explicit grounding rule for when `search_docs` is still the right tool - state only what retrieved chunks actually say, don't generalize a pattern across the whole system unless a chunk states it explicitly. Closed a real, previously-flagged gap: a "tell me about SQL migrations" question that had settled for a 4-migration `search_docs` sample and narrated unsupported claims ("cumulative," "idempotent") beyond it, caught by the verifier at the time - re-tested after the fix, correctly retrieved all 13 migrations via SQL with every claim traceable to evidence. 2026-08-07, later same day — two more real bugs, found reviewing live traffic rather than deliberate testing. First: `flatten_history()`'s last-3-rounds "full fidelity" window spliced in each round's *entire* stored `full_messages`, but `ask_agent()` had never separated a round's own new turns from the inherited history it started with, so every round's stored history already contained its own copy of the previous rounds' — compounding roughly 2x per round with no bound (traced back to the earliest surviving `chat_rounds` row: 6 → 10 → 20 → 40 → 76 → 144 → 274 → 506 → 938 messages, one question eventually costing 176,252 tokens despite a completely correct underlying query). Fixed via `own_messages_start`, tracking where a round's own turns begin so only that slice gets persisted. Second, surfaced immediately after by re-asking the exact same question against the now-clean history: `SYSTEM_PROMPT`'s routing rule never named "latest commits/activity/work" as a tracked category, so a "latest commits" question picked `docs/BUILD_7_FLOW/build7-handoff-brief.html` for sounding newest by name and reported Build 7 as the latest work — silently missing two weeks of subsequent incidents that exist only in the cheat sheet, since per-build handoff briefs are frozen the moment that build ships. Fixed by widening the trigger list and stating that rule explicitly. Full account in the deployment-verification audit's Phase 8. 2026-08-07, still later the same day — the fidelity window itself turned out to bound round *count*, not per-round *size*, a risk already named unmitigated in this feature's own original planning notes: one legitimately large tool result (58,824 chars) got replayed whole into a follow-up question's context, tripling its cost before that question ran a tool call of its own. Fixed with `MAX_HISTORY_TOOL_RESULT_CHARS = 3000` and `_cap_tool_results_for_history()`, capping what a round's `full_messages` carries forward for a future round to inherit - never what the round sees live, so correctness for the round that actually gathered the data is untouched. Verified live: an identical follow-up question dropped from 76,403 to 23,988 tokens. Retrying an earlier question then surfaced three more real bugs in one place (full account in Phase 10): a model-written SQL query missing parentheses around an OR group, silently dropping its intended `source_file` scope (SQL evaluates `AND` before `OR`) - fixed with an explicit parenthesization rule in `SYSTEM_PROMPT`; the main answer's own `max_tokens` (1024) silently truncating a genuinely long, correct answer mid-list with zero indication - raised to 2048, and a `stop_reason == "max_tokens"` check now appends an explicit cut-off notice instead of returning the truncated text as if complete. Full account of all three, and the matching `verify_answer.py` fix, in the audit's Phase 10. 2026-08-07, closing the day — the grounding rule from earlier that same day (state only what evidence actually shows) had been written specifically for `search_docs`, and never generalized: a "what type of cameras do you have?" answer correctly listed 5 item names from `run_sql_query`, then added fabricated real-world classifications ("mirrorless," "full-frame," "360-degree") the query never returned - caught precisely by the verifier, evidenced with a live screenshot. Fixed by restating the rule as tool-agnostic - "this grounding rule isn't specific to search_docs - it applies to every tool's evidence, including run_sql_query" - naming this exact case directly in the prompt text, the same way "list every X" phrasing patches eventually generalized into a category rule earlier that day. Re-tested live on a fresh rebuild: the model now lists bare item names with no added classification, verifier confirmed `supported: true`. Full account in the audit's Phase 11. 2026-08-10, later — a distinct bug from the same family, found live: a "what's in my scratch workspace" question called `list_tables`, which returns rows across `clean`, `agent_scratch`, and `docs` together (11 rows that day); the answer correctly filtered the displayed list down to the 8 real `agent_scratch` tables, but still opened with "you have 11 tables" - the tool result's raw, unfiltered row total, not the count of the subset it had just filtered down to. Every individual fact stated was accurate; the number attached to the sentence was not - a same-evidence miscount, not a fabrication beyond evidence, so the existing grounding rule didn't cover it. Fixed with a new, deliberately general rule (not scoped to `list_tables` specifically): filter a tool result down to what the question actually asked about before counting it, never state a tool result's raw row total as if it were a narrower count. Re-tested live on a fresh rebuild with the identical question: answer correctly opened with "8 scratch workspace tables," verifier confirmed `supported: true`, explicitly noting "the count of 8 tables is correct."

`src/tools.py`

Every tool the agent can call: `run_sql_query`, `list_tables`, `describe_table`, `save_dataframe`, `search_summaries`, `search_docs`, and (2026-08-10) `list_stale_scratch_tables`. `run_sql_query` has two independent safety caps: `MAX_SQL_RESULT_ROWS = 200` (row count) and, added 2026-08-06, `MAX_FIELD_CHARS = 500` (single-value size) — a query can blow the token budget past either cap alone even while respecting the other, confirmed live when a 20-row `GROUP BY summary` result (well under the row cap) still cost 70,007 tokens for one question because individual `summary` fields ran 1,500+ characters; the same fix dropped a `SELECT *` result from 55,504 to 12,051 characters by capping the `summary_embedding` vector column too. 2026-08-10 — `save_dataframe` now records a `table_metadata` row on genuine table creation (not on a later append to an existing table); `list_stale_scratch_tables(mention_after_days=7, delete_eligible_after_days=30)` reports scratch tables by age, two-tier, read-only, agent-callable; `delete_scratch_tables(table_names)` does the actual dropping but is deliberately *not* registered as an agent tool at all - it's called only from a direct Streamlit UI button click (see `app.py`), and re-verifies each table's eligibility itself before dropping anything rather than trusting the caller, the same defense-in-depth shape as `SAFE_NAME` + `sql.Identifier()` both guarding `table_name` elsewhere in this file. `list_tables`/`describe_table` extended to exclude `table_metadata` from discovery, same treatment as `chat_rounds`. Same day, follow-up — `delete_scratch_tables` gained `require_eligibility: bool = True`: the default keeps the age re-check, but `False` (used only by the new "delete any table now" sidebar section) skips it entirely for a human explicitly picking a table by name - `SAFE_NAME` and the protected-name check still apply either way. New `list_all_scratch_tables()` lists every scratch table with no age filter, backing that same section - not agent-callable, the agent already has `list_tables` for general discovery.

Python — observability & eval harness (Build 4, src/)

`src/logging_utils.py`

Structured JSON Lines tool-call logging — one line per call: timestamp, `question_id`, turn, tool name, input, output, error, latency. Build 7 — `get_tool_calls()` gained an `include_output` flag (default `False`, existing callers unaffected); new `get_verification()`/`log_verification()`, a third distinct log-entry shape keyed on `supported` so it can never collide with `get_final_answer()`'s `"answer"` filter. 2026-08-07 — new `log_verification_error()`, a fourth distinct shape keyed on `verification_error`, added after a real verifier crash was found traceable only by reproducing the exact API call by hand - the console `print()` that should have shown it had already scrolled out of the container's log buffer by the time it was investigated. 2026-08-10 — `log_tool_call()` now also records `result_chars` (`len(str(output_data))`) and `approx_tokens` (`result_chars / 4`) for every call. Not a real per-tool-call token count - the Anthropic API only reports usage per API turn, never per tool call, and a turn can make more than one tool call at once (confirmed live: a two-tool-call turn shares one `response.usage`), so there's no way to get an exact token cost for one call from the API alone. Result size is what actually gets resent as context on every later turn - the same value `str(result)` already becomes as the tool_result content in `agent.py` - so it's exactly attributable to the one call that produced it, with no cross-turn or multi-call ambiguity, the same proxy `MAX_FIELD_CHARS`/`MAX_SQL_RESULT_ROWS` already use for this identical problem. `get_tool_calls()` surfaces both fields with `.get(..., 0)` defaults so reloading a chat round logged before this existed doesn't crash.

`src/eval_cases.py / src/eval_cases_round2.py`

Fixed sets of test questions with an expected tool and expected answer keywords — run end-to-end against the live agent, not tested in isolation. Two cases fixed 2026-08-06 after real observed flakiness, not guessed: the "agent_scratch two roles" question reworded to directly ask for both role names (an open-ended "why" question let the model paraphrase around a literal identifier some runs); the stale "total revenue from orders" question (customers/orders left the agent's scope in Build 6) reframed around the grounded, correct behavior — redirecting to `clean.allocations` — instead of expecting a fabricated $ figure, with `expected_tool` now a list since list_tables/search_docs/run_sql_query are all legitimate paths to that same correct answer. 2026-08-10, later - CI run 31449449929 failed 2 of 7 eval cases on `tool_ok` alone (both had `keywords_ok: True`, both answers fully correct) - the "agent_scratch two roles" and "camera equipment" questions both used `run_sql_query` instead of their originally-expected `search_docs`/`search_summaries`, following SYSTEM_PROMPT routing rules already added 2026-08-07 ("what are the X" and individual-item/category questions). Not a regression - the last green eval run (`0ed4f11`) took the originally-expected path for the same two questions; both are legitimate given the current prompt, the eval cases were just never updated after those routing rules widened. Fixed the same way the revenue case already was: both `expected_tools` widened to lists.

`src/run_eval.py`

Drives every case through `ask_agent()`, checks the tool-call log and answer text, reports pass/fail per question. Accepts an optional case-module argument. Real bug found 2026-08-05, the first time this ran in CI after weeks of nothing reaching `origin/main`: `tools_called` assumed every new log line was a tool-call entry, but `log_final_answer/log_verification` entries (Build 6/7) share the same file and lack `tool_name` — fixed with the same `"tool_name" in entry` guard `logging_utils.py` already uses. 2026-08-06 — `expected_tool` can now be a list, not just one string, for cases where several different tools are all legitimate ways to reach the same correct conclusion.

`src/metrics_rollup.py`

Plain-text report over the tool-call log — tool-usage frequency, average turns per question (grouped by `question_id`), error rate.

Python — live deployment with CI (Build 5, src/)

`src/synthesize_dataset.py`

Builds `data/ALLOCATION-synthesized.csv` from the real export — real dates/duration/renewal/resource counts kept, department/email/summary synthesized. Imports the real department-code mapping from a git-ignored local file, never hardcodes it.

`department_mapping.local.py`

Git-ignored, repo root — the real department-code mapping `synthesize_dataset.py` actually runs from. Never committed; see `DEPARTMENT_MAPPING.local.md` for the human-readable copy.

`pytest.ini`

Repo root — `python_files = test_*.py` scopes discovery to this project's actual test-file convention, after a stray non-test script (`api_test.py`) was found being silently collected by pytest's broader default. Gained `testpaths = tests` / `pythonpath = src` in the 2026-08-05 reorg.

Python — front end for the agent (Build 6, src/ + tests/)

`src/app.py`

The Streamlit entrypoint — chat UI wrapping `ask_agent()`, hybrid tiered history threading via `session_state.rounds`, a persisted feed (Phase 3.5) loaded from `agent_scratch.chat_rounds` on first load, and an FAQ-style render: only the newest round shows expanded, older rounds collapse into `st.expander(label=the question)`. Build 7 — `render_verification_badge()` shows a green `st.success()`/red `st.error()` status right after each answer (nothing rendered when there's no verification entry at all), using Streamlit's own safe text escaping rather than raw HTML since the verdict's `reason` text is LLM-generated. 2026-08-10 — a new sidebar section is the *only* place scratch-table deletion can actually happen: lists tables via `list_stale_scratch_tables()`, splits them into "worth a look" vs. "eligible for deletion" (30+ days), and only the eligible ones get a checkbox + delete button. The result of a delete is stashed in `session_state.scratch_cleanup_result` and shown once, right after `st.rerun()`, so the success/skip message survives the rerun instead of flashing and vanishing. Same day, follow-up — a second, always-available section below a `st.divider()`: every scratch table via `list_all_scratch_tables()`, no age filter, calling `delete_scratch_tables(..., require_eligibility=False)` - for a table the user is done with immediately rather than waiting out the staleness thresholds. Its own `session_state.scratch_immediate_delete_result` key, kept separate from the eligible-tables section's result so the two don't overwrite each other's message. Same day, follow-up — `render_tool_call_details()` shows each call's `result_chars`/`approx_tokens` next to its latency, but only once a question has more than one tool call - with a single call there's nothing to isolate, that call already is the whole answer's tool cost. Requested specifically to help isolate which tool call in a multi-call question caused a token burn, after the scratch-workspace miscount incident the same day made clear that per-turn usage numbers alone don't point at one call.

`tests/test_agent_history.py`

3 tests for `flatten_history()` — the pure function behind hybrid tiered conversation memory. 2026-08-07 — 2 more added for `ask_agent()` itself, mocking `client.messages.create`: confirm a round's stored `full_messages` holds only its own new turns, and that message count stays flat across 6 simulated rounds instead of compounding. Both confirmed to fail against the pre-fix code and pass against the fix (via a temporary `git stash`, not assumed) — the existing 3 tests never caught the real bug because they exercised `flatten_history()`'s read side with fixtures that already assumed the correct write-side contract `ask_agent()` was silently violating. Later the same day — 3 more for `_cap_tool_results_for_history()` (oversized content truncated with a sentinel, small content untouched, and an end-to-end `ask_agent()` check that the live round still sees the real, uncapped result while only the persisted copy is capped), plus 1 for the answer-truncation notice, reproducing the real cut-off-mid-list bug directly.

`tests/test_chat_rounds.py`

3 tests — save/load round-trip, discovery exclusion, and `MAX_CHAT_ROUNDS`'s exact boundary via `monkeypatch`.

`src/app_demo.py`

2026-08-11 — the public Streamlit Community Cloud entry point, a deliberately separate file from `app.py` rather than a flag on it: two things that are correct for one trusted local operator (persistent shared history, a scratch-table-management sidebar) are not automatically safe for an anonymous public link, so both change here rather than being bolted on conditionally. History is session-only - never calls `load_chat_rounds()`/`save_chat_round()` at all, so a second visitor can never see a first visitor's questions, the exact privacy gap `app.py`'s global-load design would have if deployed publicly as-is. The scratch-management sidebar is removed entirely, not just hidden - a maintenance surface an anonymous visitor has no reason to see. Two cost guards, both new: a soft per-session cap (`DEMO_MAX_QUESTIONS_PER_SESSION = 15`, a `session_state` counter, resets on reload - not a security boundary, just a nudge) and a hard daily token cap shared across every visitor (`DEMO_DAILY_TOKEN_CAP = 300_000`, checked via `get_todays_demo_token_usage()` before every question, logged via `log_demo_token_usage()` after every answer) - together the same defense-in-depth shape as the SQL role boundary: a soft per-caller limit plus a hard global one, neither alone sufficient.

`tests/test_demo_guardrails.py`

2026-08-11 — 4 tests against real `agent_scratch.demo_token_log` rows: a fresh day sums to zero, logging then reading back sums correctly, a row backdated to yesterday (via the superuser connection - `appdb_agent_writer` was deliberately never granted `UPDATE`, same reasoning as `table_metadata`) is correctly excluded from today's total, and the table stays excluded from `list_tables()`/`describe_table()` discovery.

Python — verifier/critic agent (Build 7, src/ + tests/)

`src/verify_answer.py`

The verifier itself — a second, narrower agent judging whether a proposed answer is supported by the tool evidence gathered for it. Forces `tool_choice` so the verdict is always a structured `{supported, reason}`, never prose. Importable (`agent.py` uses it directly) and CLI-runnable standalone against any real logged `question_id`, with an optional override-answer argument to prove the deliberately-bad case. 2026-08-06 — `VERIFIER_SYSTEM_PROMPT` taught to check whether a per-category SQL condition is a true subset of a stricter total's condition, not just compare the resulting numbers, after a real false positive; a real false *negative* found later the same day (a correct accessory-exclusion judgment flagged as unsupported) resolved on its own once the underlying data was normalized — no further verifier changes needed. 2026-08-07 — a real crash found live: an unusually large evidence+answer payload hit this call's own `max_tokens` cap mid-generating `report_verdict`'s JSON, truncating it after `supported` but before `reason` was ever written; `block.input["reason"]` then threw `KeyError`, silently swallowed by `agent.py`'s broad `try/except` - no badge shown at all, not even red. Fixed: `max_tokens` 512 → 1024, plus `.get()` with an explicit fallback reason instead of bracket-indexing a possibly-truncated response. A response missing `supported` entirely still raises, correctly left to the caller's own safety net. 2026-08-10 — `VERIFIER_SYSTEM_PROMPT` extended to distinguish two different unbacked-claim shapes, found live: a "describe the allocations table" question (its own `describe_table` call) followed ~50 seconds later by a follow-up asking for the same columns plus a row count. The follow-up correctly reused the column list from the prior round's real evidence - still sitting in the hybrid-tiered-memory context the model actually saw - rather than re-calling `describe_table`, but this call's own evidence (`get_tool_calls(question_id, ...)`) is scoped to just this question's fresh tool calls, so the verifier flagged a genuinely correct answer as unsupported. Rather than expanding the verifier's evidence scope (real tradeoffs: cost of a growing payload, and verifying against evidence the model's own context window may not have actually included - see the cheat sheet's `agent.py` row on `MAX_FULL_FIDELITY_ROUNDS`), the fix teaches the verifier to tell two cases apart from the evidence alone, with no history access needed: a claim that directly contradicts the evidence shown stays hard, unqualified "unsupported"; a claim the evidence neither confirms nor denies gets `reason` prefixed "Verify further:" instead, naming exactly what can't be confirmed - honest either way, since this evidence-limited check genuinely can't tell a legitimate history-carryover from an outright fabrication, only tell "conflicts with what I see" from "isn't in what I see." `supported` stays a plain boolean either way (still `false`, badge stays red) - only the `reason` wording changes, so no schema/logging/UI changes were needed. Re-verified: all 5 `verify_eval_cases.py` cases still pass, including the outright-contradiction case staying hard-worded, not softened. Re-tested live on the exact real scenario: `reason` now reads "Verify further: ...which has no visibility into any earlier schema queries that may have been run." 2026-08-11 — the "Verify further:" prose-prefix convention above turned out to never actually fire in practice, found live on the public demo: asked "Tell me about this project?", the agent's own general background knowledge (true, but not present in that turn's two tool calls) bled into schema-specific claims about the equipment-checkout domain, and the verifier hard-failed them with no prefix. Reproduced directly and repeatedly (same question, same evidence, same answer, run 3x outside the app) rather than assumed: `supported=False` every time, and critically the verifier's own `reason` text kept independently describing the exact "evidence doesn't confirm this, but doesn't contradict it either" situation the prefix exists for - it just never wrote the prefix. Two prompt-wording-only fixes (disambiguating "claims something the evidence never shows" out of the hard-fail paragraph, then clarifying that confident phrasing isn't itself evidence) were tried and reproduced-tested first; neither changed the outcome across 6 total runs. Root cause: a boolean field plus a "remember to prefix your reason with this exact string" prose convention is not something structured tool-calling actually holds reliably, no matter how the surrounding rule is worded. Fixed structurally instead of textually: `report_verdict`'s `supported: bool` input replaced with `verdict: enum["supported","contradicted","unconfirmed"]`, forcing a real categorical tool-argument choice; `verify_answer()`'s Python code (not the model) now derives the caller-facing `(supported: bool, reason: str)` tuple and attaches the "Verify further:" framing when `verdict == "unconfirmed"` - the external shape every caller (`agent.py`, `app.py`/`app_demo.py` via `get_verification()`, `run_verify_eval.py`) already depended on is unchanged, so none of them needed edits. Re-reproduced against the same live scenario post-fix: verdict and reason text now stay consistent with each other every run (one run even flipped to genuinely `supported=True` - real judgment variance, not a stuck default). Re-verified: all 5 `verify_eval_cases.py` cases still pass, plus 3 new unit tests covering the `contradicted`/`unconfirmed`/unexpected-verdict-value paths.

`src/verify_eval_cases.py`

Hand-constructed evidence + answer pairs (not read from the live log, for CI-portability) proving the verifier catches a bad answer, not just that it never complains — a grounded case, an outright contradiction, a fabricated number, plus two look-alike cases added 2026-08-06 after a real false-negative: per-category SQL counts broader than the total they're compared against (should fail) vs. genuine overlapping categories that legitimately sum past a total (should pass) — same surface shape, opposite correct verdicts.

`src/run_verify_eval.py`

Mirrors `run_eval.py`'s shape — runs every case through `verify_answer()`, full per-case report, non-zero exit on any mismatch. Never wired into CI until the 2026-08-05 audit — see Claude's Commands.

`tests/test_verification.py`

2 tests for `get_verification()`'s found/not-found paths — temp log file + `monkeypatch`, no API cost for the pure-logic half of Build 7. 2026-08-07 — 2 more for `log_verification_error()`: writes a findable entry, and that entry's distinct shape (`verification_error`, not `supported`) never accidentally matches `get_verification()`'s own filter.

`tests/test_tool_call_size.py`

2026-08-10 — 4 tests, same temp-log-file + `monkeypatch` pattern as `test_verification.py`: `log_tool_call()` records the correct `result_chars`/`approx_tokens` for a real result, both are 0 on an error call (matching how `agent.py` actually calls this - `output_data=None` on failure), `get_tool_calls()` surfaces both fields correctly, and a log line written before this feature existed (missing both fields entirely) still loads via `.get(..., 0)` defaults instead of crashing.

`tests/test_verify_answer.py`

2026-08-07 — 4 tests for `verify_answer()` itself, mocking `client.messages.create`: the normal case is unaffected, a truncated `report_verdict` missing `reason` falls back instead of crashing (reproduces the real `KeyError` directly - confirmed to fail against the pre-fix code), one missing `supported` entirely still raises, and a response with no `tool_use` block at all still raises too. 2026-08-11 — updated for the `verdict` enum (fixtures now send `{"verdict": ...}`, not `{"supported": ...}`), plus 3 new tests: `contradicted` maps to `supported=False` with `reason` untouched, `unconfirmed` maps to `supported=False` with `reason` prefixed "Verify further:", and an out-of-enum verdict value degrades to unsupported rather than crashing.

`tests/test_extract_doc_chunks.py`

2026-08-10 — 5 tests for `_split_long_text()`, the pure chunk-splitting helper: short text passes through unchanged, long text splits on sentence boundaries with no content lost, every split piece stays under the live `MAX_FIELD_CHARS` (500) - checked against that real number, not just the function's own internal target - a single sentence too long to split on periods still gets hard-wrapped on word boundaries, and full word-level content survives the round trip. No DB/API cost - pure logic, same pattern as `test_verification.py`. Same day, follow-up - 3 more for `extract_list_chunks()`: feature-list items extract correctly, `dt/dd` pairs join into one chunk each, and a trailing unmatched `dt` is silently dropped rather than erroring - documents the real limitation named in that function's own comment (positional `zip()` pairing) rather than leaving it as an unproven claim.

`tests/test_scratch_cleanup.py`

2026-08-10 — 12 tests against real `agent_scratch` tables: a new table gets a `table_metadata` row, a fresh table isn't reported stale, a 10-day-old table is worth-mentioning but not eligible, a 35-day-old table is eligible, deletion refuses an ineligible table (and leaves it in place), deletion removes an eligible table's data *and* its metadata row, deletion rejects an invalid name and both protected system-table names, and `table_metadata` itself stays excluded from `list_tables`/`describe_table`. Backdating a row to simulate age uses the superuser connection, not `appdb_agent_writer` - that role was never granted `UPDATE` on `table_metadata`, on purpose (see the migration row above), so this is a real permission boundary the test respects rather than works around. Same day, follow-up — 3 more: `require_eligibility=False` deletes a genuinely 0-day-old table, that same bypass still rejects an invalid name and both protected system tables (the age gate is skippable, `SAFE_NAME` and the protected check never are), and `list_all_scratch_tables()` includes a fresh table while still excluding `chat_rounds`/`table_metadata`.

Python — data pipeline (Build 2, src/)

`src/ingest.py`

Loads the source CSV into `raw.allocations`, one `executemany` call.

`src/profile_raw_data.py`

Pandas exploration of `raw.allocations` that informed `005_clean_schema.sql`'s design. Renamed from `profile.py` — it silently shadowed Python's stdlib `profile` module.

`src/transform.py`

Raw → clean transform: type coercion, blank→NULL handling, populates `clean.allocations`.

`src/build_allocation_items.py`

2026-08-06 — parses `clean.allocations.summary` (pipe-delimited, "Returned: " prefix stripped) into `clean.allocation_items`, one row per item, via a closed `ITEM_CLASSIFICATION` lookup for all 55 distinct item names found by a full audit of the data — fails loudly on any unclassified name rather than silently miscategorizing it. Full rebuild (`TRUNCATE` + reload) each run, not incremental — the source is a closed, fully-read set of strings, not a growing feed.

Python — semantic search (Build 3, src/)

`src/embed_summaries.py`

Idempotent backfill — embeds every `summary` row still missing a `summary_embedding`.

`src/search_summaries.py`

Standalone script proving raw cosine-distance search works, before any agent wiring.

`src/calibrate_threshold.py`

Statistical calibration of the 0.63 relevance-distance cutoff, from 40 curated in-/out-of-domain queries.

Python — documentation RAG (Build 3.5, src/)

`src/extract_doc_chunks.py`

Parses this project's own docs into embeddable chunks — four extraction shapes across two design eras, plus narrative prose. `SOURCE_FILES` auto-discovers every handoff brief via `glob` (Build 6 fix) rather than a hand-maintained list, after that list twice silently fell behind a newly-written brief. Glob target updated to `docs/` in the 2026-08-05 reorg (was `PROJECT_DIAGRAMS/`). 2026-08-10 — new `_split_long_text()`, applied once centrally after all four extraction functions run: any chunk over 450 characters gets broken up on sentence (falling back to word) boundaries before it's ever stored, so no chunk can be long enough for `tools.py`'s `MAX_FIELD_CHARS` (500) to truncate anything out of it - closes the general mechanism behind a real incident where a truncated chunk's cut-off resolution text made an already-fixed problem look still open. Found in 127 of ~532 chunks before this fix (up to 6,118 chars); zero after, across all 769 post-split chunks. Separately, the whole connect-and-extract body was module-level with no `if __name__ == "__main__":` guard - wrapped in `main()` so importing the pure splitting function for a test doesn't open a live DB connection as a side effect. 2026-08-10, later — two more fixes from swapping `docs/project-overview-v2.html` into `SOURCE_FILES` in place of the original. First: `main()`'s per-path `DELETE ... WHERE source_file = %s` only ever fires for a path still in `SOURCE_FILES` - a path removed from the list entirely (renamed, replaced) keeps its old rows forever, confirmed real: the swap left 9 rows stranded under the old path with zero cleanup path, the same "orphaned rows" shape the 2026-08-05 repo audit already found once for `PROJECT_DIAGRAMS`'s old paths. Fixed with one `DELETE FROM docs.chunks WHERE NOT (source_file = ANY(%s))` run once at the start of `main()`, before the per-path loop. Second: new `extract_list_chunks()`, added after the swap silently extracted only 3 of the new file's 21 real chunks - its bullet feature lists (`.feature-list li`) and definition-list build breakdown (`.build-list dt/dd` pairs, matched positionally via `zip()`) don't land in any of the four existing shapes, the same gap `extract_narrative_chunks()` was originally built to close for plain `.lede` prose. Re-verified: 21/21 chunks extracted post-fix, confirmed live that a "what is this project" question now answers from the new content.

`src/embed_chunks.py`

Idempotent backfill for `docs.chunks` — same `WHERE embedding IS NULL` pattern as `embed_summaries.py`.

`src/search_chunks.py`

Standalone script proving raw similarity search over the docs corpus works, before any agent wiring.

Tests (tests/)

`tests/test_data_quality.py`

Build 2 — pandera schema checks against `clean.allocations`.

`tests/test_agent_scratch_boundary.py`

Build 2.5 — role-boundary tests plus the `MAX_SCRATCH_TABLES` cap test.

`tests/test_semantic_search.py`

Build 3 — embedding coverage, relevant/irrelevant query behavior, HNSW index existence.

`tests/test_docs_search.py`

Build 3.5 — same shape as `test_semantic_search.py`, applied to the documentation corpus.

Configuration & infrastructure (repo root)

`README.md`

Added in the 2026-08-05 reorg — the repo's actual entry point (what this is, quickstart, folder layout, trust boundary). Didn't exist before.

`docker-compose.yml`

Defines `pg_local`, `mssql_local`, the `agent` service, and (Build 6) `streamlit_app` — same image as `agent`, just a different `command:`. Postgres image is `pgvector/pgvector:pg17`. `streamlit_app`'s command updated to `src/app.py` in the reorg.

`.githooks/pre-commit`

2026-08-07 — tracked (not `.git/hooks/`, which never survives a clone; wired in via `git config core.hooksPath .githooks`). Re-runs `extract_doc_chunks.py` + `embed_chunks.py` whenever a commit touches the cheat sheet, project overview, or any handoff brief, blocking the commit on failure — closes the same class of gap as the deployment-verification incident, one layer down: `docs.chunks` is data, not code, so it needs its own explicit resync step, and nothing had been re-running it automatically after a doc edit. Runs on the host directly (the project's own `.venv` has every package it needs; `pg_local` is reachable at `127.0.0.1:5432`), no container involved.

`.streamlit/config.toml`

2026-08-06 — `fileWatcherType = "none"`. Originally suspected as the cause of a 36.5s latency spike (its module-tree walk produces harmless `torchvision` `ModuleNotFoundError` noise timestamped inside the slow window); disproven by testing a non-Streamlit process in the same container, which was equally slow. Kept anyway — it's a real, separate fix for the documented log-noise issue, just not the latency one (that turned out to be the missing model-cache fix, see the Dockerfile row above).

`Dockerfile`

Builds one shared image used by both the `agent` and `streamlit_app` services — differentiated only by each service's `command:` override, not two separate Dockerfiles. Build 7 — `COPY` line fixed to include `verify_answer.py`. Reorg — `COPY`/`CMD` updated to `src/` paths. 2026-08-06 — gained a `RUN` step that downloads the `all-MiniLM-L6-v2` embedding model at *build* time, after a fresh/recreated container's first embedding call was measured at 36.5s (a full network re-download) vs. 2.4s once cached; baking it into the image removes the runtime network dependency entirely rather than just caching it after the fact. Also copies in `.streamlit/config.toml`.

`requirements.txt`

Pinned Python dependencies — `anthropic`, `beautifulsoup4`, `psycopg`, `pgvector`, `sentence-transformers`, `pandas`, `pandera`, `pytest`, `python-dotenv`, `streamlit` (Build 6).

`.env`

Real local secrets (DB passwords, API key) — gitignored, never committed.

`.env.example`

Template shape of `.env`. Rewritten in the 2026-08-05 audit — the prior version documented 4 of the 10 vars the code actually requires (missing `ANTHROPIC_API_KEY` and both reader/writer role credentials entirely), meaning a fresh clone following it hit an immediate `KeyError`.

`.gitignore`

Excludes secrets (`.env`, `CREDENTIALS.local.md`, `DEPARTMENT_MAPPING.local.md`, `department_mapping.local.py`), OS/editor cruft, `__pycache__/`, logs/dumps. Personal working-style reference path updated to `docs/Working-Style_Prompt/` in the reorg.

`.github/workflows/ci.yml`

Build 5 — GitHub Actions: migrations + full pipeline + pytest on every push; a gated eval-harness job on a relevant merge to `main` only. 2026-08-05 — migration list gained `012_chat_rounds_schema.sql` (was silently missing), gained a step running the verifier's own eval harness (was never wired in at all), and every path updated for `src//migrations/`. 2026-08-06 — `013_allocation_items_schema.sql` and a new "Build normalized allocation items" step added to **both** jobs' migration lists deliberately, given the previous day's bug was exactly a migration missing from one of two duplicated lists.

`.gitattributes`

Forces consistent line endings for `.sql/.yml/.md` across Windows/Unix.

Docs, secrets, legacy & generated artifacts

`CREDENTIALS.local.md`

Repo root — unredacted local companion to `.env` — gitignored, never shared or committed.

`DEPARTMENT_MAPPING.local.md`

Repo root, Build 5 — the real department-code mapping used by `synthesize_dataset.py`, human-readable. Gitignored, same treatment as `CREDENTIALS.local.md` — publishing it would let anyone reverse the department masking.

`data/ALLOCATION-synthesized.csv`

Build 5 — the project's one official dataset (1535 rows), real temporal/quantitative fields kept, everything else synthesized. Used identically by local dev, CI, and the eval harness. Location unchanged by the reorg.

`docs/sql-test-01-cheatsheet.html / .pdf`

This living document — every command actually typed, plus this file map. Moved from the repo root into `docs/` in the 2026-08-05 reorg.

`docs/`

Renamed from `PROJECT_DIAGRAMS/` in the reorg. One subfolder per build (`BUILD_1_FLOW` … `BUILD_7_FLOW`) — flow diagrams (`.svg/.html/.png`) and handoff briefs (`.html/.pdf`). Build 1's two handoff briefs live here too, recovered from Artifacts during Build 3.5.

`docs/project-overview.html`

Build 3.5 — a seed document written to answer whole-project overview questions, so the docs-RAG corpus has something to match against. Kept unchanged on disk since 2026-08-10, when `project-overview-v2.html` (below) replaced it in `SOURCE_FILES` - deliberately left in place as the original rather than deleted.

`docs/project-overview-v2.html`

2026-08-10 — a full content rewrite of the project overview, requested explicitly as a fresh presentation for a working system rather than the original's early-stage "planned: Build 4/5" framing (four builds stale by the time this was written). Same visual design system as the cheat sheet and commit log (same CSS tokens/fonts), extended with `h2/.stats/.feature-list/.build-list` for a "what it does" / "what makes it stand out" / "how it was built" structure - grounded in real, checked numbers (7 builds, 14 migrations, 60 tests, CI-gated, 3 documented incidents) rather than adjectives. Swapped into `SOURCE_FILES` in place of the original (see `extract_doc_chunks.py`'s row for the two extraction-pipeline fixes that swap required).

`logs/`

Build 4 — gitignored directory holding `tool_calls.jsonl` (live, JSON Lines) and `tool_calls.pre-question-id.jsonl` (archived). Durable, queryable tool-call tracing — not a Postgres table, deliberately. Location unchanged by the reorg (still repo root, matching the `./logs:/app/logs` volume mount).

`legacy/schema_mssql.sql`

The SQL Server counterpart to `001_schema.sql` — same `appdb` shape, T-SQL dialect. Moved to `legacy/` in the reorg — unreferenced by any code, CI, or Docker config in this project; part of the original dual-database lab, not the seven-build agent narrative.

`legacy/seed_mssql.sql`

SQL Server counterpart to `002_seed.sql`. Same disposition as `schema_mssql.sql` above.

`.vscode/, .pytest_cache/`

Editor settings and pytest's own cache — auto-generated, not source.

Docker

CONTAINER ORCHESTRATION

Starting the stack, and getting SQL in and out of a running container.

```
docker compose up -d --build
```

Build images if anything changed, then start every service in the background.

```
docker ps
```

List running containers — the first check when a connection unexpectedly fails.

```
docker exec -it pg_local psql -U devuser -d
appdb
```

Open a **live, interactive** psql session inside the Postgres container.

```
Get-Content file.sql -Raw | docker exec -i
pg_local psql -U devuser -d appdb
```

Run a whole .sql file inside the container **non-interactively**, PowerShell-style — < redirection doesn't exist in PowerShell, so the file gets piped in instead. (Bash/cmd would use `docker exec -i ... < file.sql`.)

```
docker exec pg_local psql -U devuser -d appdb
-c "SELECT ..."
```

Run a single one-off SQL command non-interactively, no piped file needed. **No `-it`** — that flag requires a real terminal attached to stdin and fails with `"cannot attach stdin to a TTY-enabled container"` in non-interactive contexts (e.g. run from a script/tool rather than a live shell).

```
docker compose down -v
```

Stop everything and delete volumes too — next start re-initializes DB passwords from .env. Used deliberately, not casually.

Build 6 — `COPY` bakes source in at build time, with no hot-reload path at all. Editing `agent.py`/`tools.py`/`app.py` on the host has zero effect on an already-running container, no matter what — stricter than the local Streamlit dev-server case, which at least auto-reruns `app.py` on save. A real `docker compose build` + up (recreating the container) is required every time, confirmed directly via `docker exec ... grep` against the container's own copy of a file rather than trusting the build log alone.

psql

META-COMMANDS & DDL PATTERNS

Once you're inside a psql session — both the backslash meta-commands and the recurring SQL shapes.

```
\d table_name
```

Describe a table — every column, its type, and any constraints. The one check that settles "did my CREATE TABLE actually work."

```
\du role_name
```

Describe a role — confirms it has no Superuser/Createrole/Createdb it shouldn't.

```
\pset null 'marker'
```

Display NULLs as a visible marker instead of blank — otherwise a NULL and an empty string look identical.

```
\! cls
```

Shell out and run an OS command without leaving psql — `cls` clears the terminal screen.

```
\q
```

Quit psql, back to the shell prompt.

```
CREATE SCHEMA IF NOT EXISTS name;
```

Create a schema, skip quietly if it's already there — safe to rerun.

```
DROP TABLE IF EXISTS name;
```

Drop a table, skip quietly if it doesn't exist — what makes a rebuild script rerunnable from a blank database.

```
GRANT SELECT ON ALL TABLES IN SCHEMA s TO
role;
```

Give a role read access to every table **currently** in a schema.

```
ALTER DEFAULT PRIVILEGES IN SCHEMA s GRANT
SELECT ON TABLES TO role;
```

Extend that same grant automatically to any table created **later** — without this, new tables start private.

```
col TYPE GENERATED ALWAYS AS (expr) STORED
```

A column Postgres computes and locks from other columns in the same row — structurally impossible for it to drift out of sync.

```
GRANT USAGE, CREATE ON SCHEMA s TO role;
```

Let a role create objects inside a schema, not just read from it — how `appdb_agent_writer` got write access scoped to `agent_scratch` only, never anywhere wider.

```
ALTER DEFAULT PRIVILEGES FOR ROLE owner IN
SCHEMA s GRANT SELECT ON TABLES TO role;
```

Extend a **different** role's read access to tables a specific owner role creates later in that schema — how `appdb_reader` can always see whatever `appdb_agent_writer` saves, without a re-grant each time.

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Enable pgvector for the database — requires the `pgvector/pgvector` Postgres image, not the plain `postgres` one.

```
ALTER DATABASE db REFRESH COLLATION VERSION;
```

Clear the collation-version mismatch warning that appears after swapping to an image built against a different glibc/ICU — safe here since the data is plain ASCII text.

```
col vector(384)
```

pgvector's column type for a fixed-dimension embedding — 384 matches the sentence-transformers model in use, not an arbitrary choice.

```
a ⟺ b
```

Cosine distance between two vectors — smaller means more similar. What `search_summaries` orders by.

```
a ⟷ b
```

Euclidean (L2) distance between two vectors — used in this project only for the self-distance sanity check (`v <-> v` should be exactly 0).

```
vector_dims(col)
```

The dimensionality of a stored vector — a quick sanity check that every row actually has a real 384-dim embedding, not a malformed one.

```
CREATE INDEX ... USING hnsw (col
vector_cosine_ops)
```

Build an approximate-nearest-neighbor graph index matching the `<=>` operator, so similarity search can use an index scan instead of a full table scan as the corpus grows.

```
EXPLAIN ANALYZE SELECT ...
```

Confirms whether the planner actually used a new index (`Index Scan`) or fell back to `Seq Scan` — the only real way to verify an index is doing anything.

PowerShell

HOST-SIDE SHELL

Running Python, checking your own environment, and vetting/installing system-level tools from outside any container.

```
.\.venv\Scripts\python.exe script.py
```

Run a script with the project's own virtual-env interpreter directly — sidesteps PATH/activation issues entirely.

```
.\.venv\Scripts\python.exe -m pip install pkg
```

Install a package into that specific virtual env, not wherever PATH happens to point.

```
.\.venv\Scripts\python.exe -m pip show pkg
```

Check whether a package is installed, and exactly which version.

```
Get-ChildItem Env: | Where-Object { $_.Name -
like "*X*" }
```

List environment variables whose name matches a pattern — e.g. catching a stray API key leaked into the shell.

```
Get-Content file
```

Print a file's contents — the go-to way to verify what actually got saved to disk.

```
claude auth status
```

Confirm this Claude Code session is on subscription login, not silently metered API billing.

```
winget show --id pkg -e
```

Pull a package's full manifest — publisher, source URL, license, install hash — before trusting or installing it.

```
winget install --id pkg -e
```

Install a package system-wide once it's been vetted, not before.

Git

CHECKPOINTING PROGRESS

Committing at the end of every phase, per the project's own working style.

`git status`

What's changed, staged, or untracked, right now.

`git add file`

Stage a specific file by name — never a blind `-A`.

`git commit -m "..."`

Record the staged snapshot with a message explaining **why**, not just what.

`git mv old new`

Rename a tracked file while git keeps its history attached to the new name.

`git log --oneline -n`

Recent commit history, one line each — the fastest way to check "where are we, really."

`git diff`

Exact unstaged changes, line by line, before they get staged.

`git remote add origin url / git push --set-upstream origin main`

Build 5 — wire up a GitHub remote for the first time and push with tracking, so plain `git push` works afterward.

`git stash push -u -m "..." / git stash pop`

Build 5 — shelve in-progress uncommitted work (including untracked files, via `-u`) to get a clean tree before a history rewrite, then restore it after. 2026-08-07 — reused without `-u`, scoped to one file (`git stash push -- src/agent.py`), to temporarily revert a fix and prove its new regression tests actually fail against the old code, not just pass vacuously against the new one — popped immediately after to restore the fix.

`git commit --amend`

Build 5 — rewrite the tip commit in place. Safe only when that commit was **never pushed**; used to fix a real mistake (real data hardcoded in a script) before it ever left the machine.

`git bundle create out.bundle --all / git bundle verify out.bundle`

Build 5 — a full, portable backup of every ref in the repo in one file. Taken before every `git filter-repo` pass this build, saved outside the repo itself.

`git log --all -S"string" --oneline`

Build 5 — pickaxe search: commits where a literal string's occurrence count changed. The actual verification method used to confirm a purge worked, not assumed from the tool's own success message.

`git show <commit>:<path>`

Build 5 — print a file's exact content as of one specific commit, without checking it out — used to directly confirm a blob no longer contains something it used to.

```
git reflog expire --expire=now --all && git gc
--prune=now
```

Build 5 — force-expire the reflog and immediately garbage-collect, so an amended-away commit's old blob is actually gone locally, not just unreferenced.

```
git push --force origin main
```

Build 5 — overwrite the remote's history after a `filter-repo` rewrite. Only used after explicit confirmation, and only because the rewrite genuinely changed already-pushed commits.

GitHub CLI

GH · BUILD 5

Installed via `winget` mid-build (not present by default) — authentication and repo creation still required the user directly; everything past that ran from this session.

```
gh auth login / gh auth status
```

Authenticate gh to a GitHub account (interactive — requires the user's own browser/credentials) and confirm the current login, protocol, and token scopes.

```
gh repo create name --public --source=. --
remote=origin --push
```

Create a new GitHub repo from the current local repo and push it in one step — the intended one-liner, not usable here since `gh` wasn't installed yet at that point.

```
gh secret set NAME --repo owner/repo
```

Set a GitHub Actions secret, prompting for the value securely (never echoed, never logged) — how `ANTHROPIC_API_KEY` got configured, by the user directly.

```
gh run list --limit n
```

Recent Actions runs for the current repo — status, workflow, branch, duration.

```
gh run view <id>
```

Step-by-step breakdown of one run — which steps passed, which failed, without opening a browser.

```
gh run view <id> --log-failed
```

Full log output for only the failed step(s) — the actual diagnostic step every real CI failure this build was root-caused from.

```
gh run watch <id>
```

Streams a run's live progress until it completes — used after every push this build to confirm a fix before moving on, rather than polling manually.

Python — psycopg

TALKING TO POSTGRES

The driver behind every script in this project — agent.py, tools.py, ingest.py, profile.py.

```
psycopg.connect(host=, dbname=, user=,
password=)
```

Open a connection to a Postgres database.

```
with conn.cursor() as cur:
```

Open a cursor scoped to a `with` block — closes itself automatically, even on error.

```
cur.execute(sql, params)
```

Run one SQL statement, with parameters passed safely (never string-formatted into the query).

```
cur.executemany(sql, rows)
```

Run the same statement once per row — how ingest.py loads 1,535 rows in one call.

```
cur.fetchall()
```

Pull every result row back as a Python list.

```
cur.description
```

Metadata about the result's columns — used to grab column names generically.

```
conn.commit()
```

Make written changes permanent.

```
conn.close()
```

Release the connection.

Python — psycopg.sql

SAFE DYNAMIC IDENTIFIERS

save_dataframe's guardrail for building a CREATE TABLE/INSERT statement where the table and column names themselves come from the model, not just the values — %s parameters alone can't protect an identifier.

`from psycopg import sql`

The submodule that builds SQL safely out of parts psycopg can't treat as plain data — like table and column names.

`sql.Identifier(name)`

Safely quotes a dynamic identifier (table/column name) — the equivalent of a parameter, but for names instead of values.

`sql.SQL(" … {} … ").format( … )`

Builds a SQL string from literal fragments plus safely-escaped pieces like `Identifier` — never plain `f"..."` string interpolation.

`sql.SQL(", ").join(parts)`

Joins a list of `Identifier`/`SQL` fragments with a separator — how a variable-length column list gets assembled.

`sql.Placeholder()`

A %s value placeholder built the same composable way, for when the number of columns isn't known until runtime.

Python — pandas

PROFILING RAW DATA

Everything profile_raw_data.py used to understand raw.allocations before designing the clean schema. (Renamed from profile.py in Build 3 — it was silently shadowing Python's stdlib profile module.)

`pd.DataFrame(rows, columns=cols)`

Build a DataFrame straight from plain rows + column names — no SQLAlchemy required.

`df["col"].unique()`

Every distinct value in a column, once each — printed with `repr()` to expose hidden whitespace.

`df["col"].value_counts(dropna=False)`

Count of each distinct value, including blanks/NaN.

`pd.to_datetime(s, format=, errors="coerce")`

Parse a column of date strings against an exact format; anything that doesn't match becomes `NaT` instead of crashing the run.

`series.dt.total_seconds()`

Convert a time difference into a plain number of seconds.

`series.str.fullmatch(pattern)`

Test every value in a column against a regex at once — used to confirm two columns were digit-only.

Python — pandera

DATA-QUALITY SCHEMAS

test_data_quality.py uses this to declare what "correct" looks like for clean.allocations, as data instead of ad hoc checks.

`import pandera.pandas as pa`

The explicit pandas-backend import for modern pandera versions.

`pa.DataFrameSchema({ ... })`

Declare an entire DataFrame's expected shape — one entry per column — as data, not scattered asserts.

`pa.Column(type, checks, nullable=)`

One column's expected type, whether NULLs are allowed, and any constraints.

`pa.Check.ge(0)`

A reusable constraint — here, "greater than or equal to 0" — attached to a Column.

`schema.validate(df)`

Check a real DataFrame against the schema; raises a real error the moment something doesn't match.

pytest

RUNNING THE TEST SUITE

First automated test suite in the project — codifies checks that were previously done by hand.

`.\.venv\Scripts\python.exe -m pytest -v`

Discover and run every `test_*` function in the project, printing each one's name and pass/fail individually.

`def test_something():`

Any function named `test_...` in a `test_*.py` file is auto-discovered — no registration needed.

`assert condition`

Plain Python `assert` — pytest reports exactly which assertion failed and the actual vs. expected values.

`with pytest.raises(ExceptionType):`

Asserts a block **must** raise a specific exception — used to prove a role boundary actually blocks a write, not just that it "seems fine."

```
def test_x(monkeypatch):
monkeypatch.setattr(mod, "NAME", val)
```

Temporarily overrides a module-level constant for one test only, auto-restored after — how a cap like `MAX_SCRATCH_TABLES` gets tested at its boundary without creating 50 real tables.

pgvector & embeddings

SENTENCE-TRANSFORMERS · PGVECTOR.PSYCOPG

Turning free text into vectors and getting Postgres to store/compare them — `embed_summaries.py`, `search_summaries.py`, `tools.py`'s `search_summaries`.

```
from sentence_transformers import
SentenceTransformer
```

A local, free embedding model library — no API key or per-call cost, unlike Voyage AI/OpenAI embeddings.

```
SentenceTransformer("all-MiniLM-L6-v2")
```

Loads the specific 384-dim model used throughout this project — first call downloads and caches it locally.

```
model.encode(text_or_texts,
show_progress_bar=True)
```

Turns one string or a list of strings into embedding vector(s) — a progress bar matters when embedding 1,500+ rows at once.

```
from pgvector.psycopg import register_vector
```

Teaches a psycopg connection how to send/receive pgvector's `vector` type directly as numpy-like arrays, instead of raw strings.

```
register_vector(conn)
```

Applies that registration to a specific connection — needed once per connection, before any vector query runs.

```
cur.execute("... WHERE col ⟺ %s < %s ...",
(embedding, threshold))
```

Passes a computed embedding straight in as a query parameter — the distance operator and the threshold cutoff both happen inside the SQL, not in Python after fetching everything.

Python — BeautifulSoup

HTML PARSING & CHUNK EXTRACTION

Turning this project's own handoff briefs/cheat sheet into embeddable chunks — `extract_doc_chunks.py`, Build 3.5.

`from bs4 import BeautifulSoup`

The HTML parsing library — reads real markup instead of treating a doc as one flat blob of text.

`BeautifulSoup(html_text, "html.parser")`

Parses a string of HTML into a navigable tree, using Python's built-in parser (no extra system dependency like `lxml`).

`soup.select(".callout")`

CSS-selector search — returns every element matching a class/tag, the same selector syntax as the CSS already written for these docs.

`soup.select_one(".fn")`

Like `select`, but returns just the first match (or `None`) — used when a chunk has at most one label element.

`element.get_text(" ", strip=True)`

Pulls plain text out of an element and its children, joining pieces with a space and trimming whitespace — turns nested markup into one clean string to embed.

Python — standard library

CSV · OS · DOTENV

No extra install required — these ship with Python itself, or python-dotenv.

```
csv.DictReader(f)
```

Read a CSV where each row comes back as a `{column: value}` dict — chosen over pandas for ingestion specifically so nothing gets silently type-coerced.

```
open(path, newline="", encoding="utf-8-sig")
```

Open a file safely for CSV reading, transparently stripping a leading BOM if present.

```
os.environ["KEY"]
```

Read a required env var — raises immediately if it's missing, rather than continuing with a silent gap.

```
os.environ.get("KEY", default)
```

Read an optional env var, falling back to a default (e.g. `POSTGRES_HOST` defaulting to `127.0.0.1` outside Docker).

```
load_dotenv(encoding="utf-8-sig")
```

Load .env into the process environment — BOM-safe, matching how Windows tools often save that file.

```
json.dumps(entry, default=str)
```

Serialize a dict to a JSON string, falling back to `str()` for anything the default encoder can't handle (e.g. `datetime`) — how `logging_utils.py` keeps every logged line valid JSON regardless of what a tool result contains.

```
open(path, "a") as f:
f.write(json.dumps(entry) + "\n")
```

The JSON Lines pattern — one independent JSON object per line, appended, never rewriting the whole file. Read back with `json.loads(line)` per line, not one big `json.load()`.

```
uuid.uuid4().hex[:12]
```

A short random ID with no coordination needed across processes — how each `ask_agent()` call gets a distinct `question_id`. Not a counter: a shared counter would itself need synchronization once calls can run concurrently.

```
time.perf_counter()
```

A monotonic, high-resolution clock for measuring elapsed time — call before and after, subtract, multiply by 1000 for milliseconds. Not `time.time()`, which can jump if the system clock adjusts.

```
sys.argv[1] if len(sys.argv) > 1 else default
```

Read an optional CLI argument, falling back to a default — how `run_eval.py` picks which eval-case module to run.

```
importlib.import_module(name)
```

Import a module by its name as a string, decided at runtime — lets `run_eval.py` load `eval_cases` or `eval_cases_round2` (or any future round) with no if/elif import chain.

```
from collections import Counter;
Counter(items).most_common()
```

Count occurrences of each distinct value and list them ranked most-to-least frequent — `metrics_rollup.py`'s tool-usage frequency.

```
n = len(open(path).readlines()); ... ;
f.readlines()[n:]
```

Checkpoint-based tailing — count existing lines before an action, then re-read only the lines appended after. How `run_eval.py` isolates one `ask_agent()` call's new log entries without `ask_agent()` needing to return that data itself.

Streamlit

BUILD 6 · APP.PY

Everything `app.py` uses to turn `ask_agent()` into a real chat interface — session state, chat primitives, and the Phase 4 tool-call expander.

```
streamlit run src/app.py --
server.address=0.0.0.0
```

Starts the dev server. The address flag is only needed inside Docker — Streamlit defaults to binding `localhost` only, unreachable through a container's port mapping otherwise; not needed for a plain local run.

`st.session_state`

A dict-like store that survives Streamlit's full-script rerun on every interaction — a plain Python variable would reset to its initial value every time. Holds live Python objects directly; no JSON serialization needed within one session.

```
if "key" not in st.session_state:
st.session_state.key = default
```

The standard init-once guard — runs on every rerun, but only actually sets the default the first time.

`st.chat_message(role)`

A styled message bubble for a given role (`"user"`/`"assistant"`) — used as a `with` block, contents render inside it.

`st.chat_input(placeholder)`

The bottom-anchored chat text box — returns the typed text once submitted, otherwise `None`.

`st.spinner("...")`

Shows a loading indicator for the duration of a `with` block — wraps the `ask_agent()` call so the wait for a real API round-trip is visibly acknowledged.

`st.error(msg)`

A visually distinct red message box — used to surface `ask_agent()`'s error field separately from a normal answer, not swallowed.

`st.expander(label)`

A collapsible section, closed by default — the `Tools used (N)` block per answer, keeping tool-call detail available without it dominating the page.

`st.code(text, language="json")`

A syntax-highlighted, monospace code block — used for each tool call's formatted `input`.

`st.caption(text)`

Small, muted secondary text — the `PROCESS_ID` line under the title, and each tool call's latency.

Build 6 Phase 3.5 — `st.expander` cannot nest inside another `st.expander`. Hit while building the FAQ-style collapsed-round view (each older round wraps its answer in an expander labeled with the question) — a round's `Tools used` detail (itself normally an expander, Phase 4) has to render as plain inline text/captions instead whenever it's inside an already-collapsed round, and only get the real widget when that round is the one currently shown expanded.

Anthropic SDK

BUILD 7 · VERIFY_ANSWER.PY

One genuinely new API pattern this build — forcing a specific tool call instead of letting the model choose freely.

```
tool_choice={"type": "tool", "name":
"report_verdict"}
```

Forces the model to call one specific tool, every time — no `stop_reason == "end_turn"` text-only path exists. How the verifier's output is guaranteed structured (`{supported, reason}`) instead of prose that would need parsing.

Claude's Commands

DIAGRAMS · VERIFICATION · PUBLISHING

A separate running list from everything above — not what *you've* typed, but the commands Claude actually runs behind the scenes while building diagrams, verifying renders, and publishing docs for this project. Appended to as new patterns come up, same as every other section.

2026-08-06 — full incident audit: the same day's token-count feature request led to a verifier false positive, a 36.5s latency regression, a single 177,090-token wrong answer, four prompt-fix iterations, and a root-cause data normalization — documented question-by-question, with real timestamps, in 2026-08-06-token-burn-incident-audit.html.

2026-08-07 — deployment verification gap, plus eight same-day follow-ons (RESOLVED - all nine parts closed same-day, full timeline below): the very fix that closed the incident above was correct and passed CI, then never reached the running browser app for nearly 20 hours — a long-running Streamlit process doesn't re-import code that changes on disk, so every `docker exec python -c` test and CI run (both fresh-process by nature) proved the fix without ever proving the deployed service had it. Cost one real question 270,961 tokens, the largest single-question total logged in this project. Fixed, then asked directly whether the underlying `docs.chunks` refresh step could be made automatic — added a tracked `.githooks/pre-commit` hook (see the file row below) so the search index can't silently go stale again. Testing that hook end-to-end immediately surfaced a second, distinct gap: semantic search ranked an older, thematically-similar document above a genuinely newer one on a "what's the latest X" question, since embedding similarity has no concept of chronological order — fixed the same way "list every X" already was, by steering that question shape toward a direct SQL query instead. That fix's own narrow phrasing-based trigger then pointed at the real, broader rule underneath it, closing a third gap: a "tell me about SQL migrations" question that had earlier settled for an incomplete 4-migration `search_docs` sample and narrated unsupported claims beyond it (flagged at the time, deliberately left open) — the SQL-routing rule widened from matching phrasing to matching category, plus an explicit grounding rule against generalizing past retrieved evidence. Two more turned up later the same day, found reviewing live traffic rather than deliberate testing: a tiered-memory bug that had been silently compounding stored conversation history roughly 2x every round since the feature's own first use (one question eventually cost 176,252 tokens despite a completely correct query); and a routing gap where the very next attempt at that same question picked a frozen per-build snapshot over the one continuously-updated file, confidently reporting months-old work as current. Two more closed out the day: the fidelity window that fixed the compounding bug turned out to bound round count, not per-round size — a risk already named, unmitigated, in this feature's own original planning notes — so one legitimately large tool result still got replayed whole into a follow-up question's context; and retrying an earlier question surfaced three real bugs in one place — an unparenthesized SQL `AND`/`OR` mix silently searching the wrong file scope, a main-agent answer silently truncated mid-list, and a verifier that crashed on an oversized

payload and showed no badge at all, its only trace a console line already lost to log rotation by the time it was investigated. An eighth and final follow-on closed the day: the grounding rule fixed hours earlier for `search_docs` resurfaced on `run_sql_query` — a correct camera list wrapped in fabricated real-world specs, caught precisely by the verifier — closed by generalizing the rule to name every evidence-gathering tool rather than patching the symptom a second time. Full account, all parts, in 2026-08-07-deployment-verification-incident-audit.html — see also the `docker compose up -d --force-recreate` row below.

2026-08-10 — two stale status claims, then a self-inflicted cost regression (RESOLVED): a routine verification pass found two live answers confidently calling the already-resolved 2026-08-07 deployment gap "still unresolved" — root cause was `MAX_FIELD_CHARS` truncating the source chunk at 500 characters, right before the word "Fixed." The first fix (a caution rule plus a front-loaded status marker) restored correctness but exploded cost to 202,821 tokens for one question - a soft "keep searching if unsure" instruction reintroducing the exact anti-pattern the 2026-08-06 incident had already named and moved away from, three days later, in an unrelated fix. Caught by re-testing before being called done, not after shipping. Closed by resolving it as data instead: a consolidated **Known Open & Deferred Items** table (below) plus a flat, non-exploratory prompt rule - but that table only fixed the two questions actually tested, not the mechanism. Called out directly: 127 of ~532 chunks already exceeded the 500-char cap (up to 6,118 chars) before this fix - the deployment-gap entry was one instance of a general problem in `extract_doc_chunks.py`'s chunking, which had no size limit at all. Real fix: chunk-splitting moved to extraction time - any chunk over 450 chars now gets broken up on sentence (or word) boundaries before it's ever stored. Re-ran the real extraction: 532 chunks became 769, zero exceed 500 characters, dataset-wide. Final re-test, post-extraction: both questions correct at 15,491 and 18,992 tokens - cheaper still. Full account, both fixes, in 2026-08-10-stale-status-claim-incident-audit.html.

```
msedge --headless --disable-gpu --window-
size=W,H --screenshot=out.png
file:///path.html
```

Renders an HTML file exactly as a browser would and saves it as a PNG — the first of two required checks before any diagram or doc is considered verified.

```
msedge --headless --disable-gpu --print-to-
pdf=out.pdf --print-to-pdf-no-header --no-pdf-
header-footer file:///path.html
```

Renders the same page through Chromium's separate print pipeline — a genuinely different code path that has caught real bugs (`<img>` references silently failing to render) the screenshot path alone missed.

```
pdftoppm -png -r 100 file.pdf prefix
```

Rasterizes every page of a generated PDF into numbered PNGs, so each page can actually be looked at instead of trusting file size alone.

```
pdfinfo file.pdf | grep -i pages
```

Quick page-count check before rasterizing — confirms whether a doc paginated the way it was expected to.

```
git commit -m "$(cat <<'EOF' ... EOF)"
```

Heredoc commit-message convention — explains **why** a change happened in the body, and avoids hand-escaping quotes/newlines. No `Co-Authored-By` trailer as of 2026-08-04 — the full history was rewritten via `git filter-repo` to strip it from all commits ahead of public/recruiter-facing use, and it's no longer appended going forward.

```
Image.open(path).convert("RGB").getpixel((x,
y))
```

Samples an exact on-screen pixel's RGB value with Pillow — confirms a color rendered exactly as CSS specified, rather than trusting a visual impression at small/compressed scale. Caught a false alarm (anti-aliasing fringe mistaken for a real color bug) during this cheat sheet's own verification.

```
WebFetch(url="claude.ai/code/artifact/<uuid>",
... )
```

For Artifacts the user owns, this returns full raw HTML rather than the usual markdown conversion — used to recover Build 1's two handoff briefs, which had only ever been published as Artifacts, never saved as local files.

```
cygpath -m "$(pwd)/file.html"
```

Converts a Git Bash path to a Windows-style forward-slash path (`C:/...`) — necessary before building a `file:///` URL for headless Edge, since Git Bash's own `/c/...` path style isn't a valid Windows path and silently fails to load.

```
git log --oneline <hash>^..<hash>
```

The exact commit range between two specific commits (the leading `^` makes it inclusive of the first one) — used to pin down a build's full commit range for its handoff brief, rather than `-n`'s "last N commits."

```
pdftotext file.pdf -
```

Build 5 — extracts a PDF's actual text to stdout. Used specifically because a plain `grep` on a PDF's raw bytes can silently miss encoded/compressed text — the real check after redacting something from a rendered document.

```
gh run view <id> --log-failed
```

2026-08-05 — a real CI failure email led here: 23 commits (all of Build 6 Phase 2 onward) had been sitting local-only, never pushed, so CI had never actually run against any of it until a history rewrite finally pushed everything. Root cause was `ci.yml`'s migration list silently missing `012_chat_rounds_schema.sql` — same bug class as `extract_doc_chunks.py`'s old hand-maintained `SOURCE_FILES` list. Fixed, then watched a fresh `gh run watch` go green before trusting it.

```
git mv old/path new/path
```

2026-08-05 — used for the full repo reorg (`src/`, `migrations/`, `tests/`, `docs/`, `legacy/`), preserving each file's git history as a rename rather than a delete+add. The same CI failure that surfaced the migration-list bug prompted a full audit, which also found `.env.example` documenting 4 of the 10 required env vars and no `README.md` at all — both fixed in the same pass.

```
grep -c '"tool_name"' logs/tool_calls.jsonl |
sort by result size
```

2026-08-06 — a new token-usage counter (in the console/UI, not a new log field on its own) surfaced a real question that cost 70,007 tokens. Rather than guess, grepped every tool result's serialized size across the whole log: 31 past results exceeded 5,000 characters, one hit 936,551 — almost certainly the exact cause of an already-documented Build 6 outage ("prompt too long: 448,866 tokens"). The proximate cause was never row count (already capped at 200) — it was individual field size (a pipe-delimited `summary` value, or a `vector(384)` embedding column pulled in by `SELECT *`), which nothing capped at all. Confirms the general lesson: when a metric jumps, pull the real distribution across history before proposing a fix — one data point can mislead, a full sweep usually can't.

```
for i in 1 2 3; do python run_eval.py; done
```

Re-running an eval case multiple times in a row before trusting a pass or a fail — a substring check on LLM-generated prose is not deterministic by construction, so one run of either result proves nothing on its own. Caught two real, different classes of flakiness this way 2026-08-06: a case whose correct answer sometimes phrased around a required literal identifier (fixed by making the question more directive, not by declaring the check broken), and a case whose correct answer legitimately reaches the same conclusion via more than one tool (fixed by allowing `expected_tool` to be a list). A third run in this same loop surfaced an unrelated real bug (a `UnicodeEncodeError` crash) that a single run would never have shown at all.

```
docker exec <container> python -c
" ... search_summaries('camera') ... " (fresh
process, no Streamlit)
```

2026-08-06 — isolated a 36.5s latency spike from Streamlit itself by running the identical embedding call in a plain, non-Streamlit process inside the same container. Still 36.2s — directly disproved the first hypothesis (Streamlit's file watcher) rather than accepting a plausible-looking correlation. A second fresh process, run after the first had already warmed the model cache, measured 2.4s — pointed straight at the real cause (no persistent cache, full network re-download on every fresh container).

```
Python: read every summary value directly,
split on '|' then ' - ', tally with
collections.Counter
```

2026-08-06 — audited the full universe of distinct equipment item names (55, across 1,535 allocations) by parsing every row in plain Python rather than trusting any single SQL `ILIKE` guess. Found the "CAMERA CASE contains CAMERA" trap recurs across at least 7 other product families, plus a second bug (real cameras inconsistently named "... BODY" vs "... CAMERA"). This audit became the classification source for `src/build_allocation_items.py`.

```
docker ps --filter name=<container> --format "
{{.CreatedAt}}"
```

2026-08-07 — the real check after deploying a fix to `streamlit_app`, not `docker exec ... grep` against the source file. A long-running Streamlit process imports its dependencies once at startup; a file changing on disk afterward has zero effect on it, so a fresh `docker exec python -c` test (which re-imports from disk every call) or a green CI run (which always builds its own fresh container) can both pass while the actual browser-facing service is still serving code from the last real `docker compose up -d --force-recreate` - in this case, nearly 20 hours stale, which cost one real question 270,961 tokens before this check caught it. `CreatedAt` older than the fix's own commit means the running process hasn't seen it, full stop.

```
git config core.hooksPath .githooks
```

2026-08-07 — the one-time setup step that activates a *tracked* git hook. Plain hooks live in `.git/hooks/`, which is never committed and so never survives a fresh clone - pointing git at a tracked directory instead means `.githooks/pre-commit` (see the file row above) actually ships with the repo and works for anyone who clones it, not just this machine.

> **Headless Edge sometimes needs a command run twice.** The first invocation can exit with code 2 and no output file at all; an identical second call then succeeds. Observed repeatedly across both screenshot and print-to-pdf renders — not project-specific, a Windows/Chromium quirk. Always confirm the output file actually exists (and, for PDFs, that `pdfinfo` reports the expected page count) after a render — never assume the first call worked.

Known Open & Deferred Items

2026-08-10 — CONSOLIDATED, CURRENT STATUS

Every item this project has ever flagged as open, deferred, or a known follow-up, in one place, kept current — not scattered across incident narratives where a status question has to guess the right keywords and hope the resolution wasn't cut off by a truncated field. Confirmed live 2026-08-10: without this table, "list all open and deferred items" cost 202,821 tokens

hunting across a dozen queries and still needed a second pass to be sure; the two real answers below are what it was actually looking for.

`Deployment verification gap`

RESOLVED — 2026-08-07, all nine parts closed same-day (fix, pre-commit hook, recency-routing gap, history-compounding bug, fidelity-window size cap, three retry bugs, grounding-rule generalization). Confirmed still holding 2026-08-10: the running container's code matches git HEAD exactly, verified by direct comparison, not inferred from timestamps. The underlying discipline — always verify a deploy via `docker ps CreatedAt`, never trust file content or CI alone — is permanent operating practice, not an open item itself.

`docs.chunks orphaned rows (PROJECT_DIAGRAMS/... keys)`

RESOLVED — confirmed 2026-08-10 via direct query (`SELECT COUNT(*) FROM docs.chunks WHERE source_file LIKE 'PROJECT_DIAGRAMS%'` returns zero). Most likely cleared by a local volume reset sometime after the 2026-08-05 finding, not the originally-suggested manual `DELETE`. See the repo-audit handoff brief for the full correction.

`Verifier blocking/retry behavior`

CLOSED, DEFERRED BY DESIGN — reconsidered and reconfirmed 2026-08-10 (not just left untouched). Considered retry-with-reason-fed-back, hedge-only, and full withholding; declined all three. The verifier's existing `reason` text already lets a user decipher what the correct answer should have been - cheap and functional as-is, and none of the three alternatives were judged worth their added complexity. A `supported: false` verdict still only logs and shows a red badge with that reason; the main answer is never retried, hedged, or withheld.

`Scratch tables cleanup policy`

CLOSED 2026-08-10 — age-based tracking and cleanup built. `agent_scratch.table_metadata` (migration 014) records each scratch table's `created_at` on creation; `list_stale_scratch_tables()` reports tables 7+ days old as "worth mentioning" and 30+ days old as "eligible for deletion" (agent-callable, read-only). Deletion is deliberately never agent-driven - `delete_scratch_tables()` is only ever called from a real button click in the Streamlit sidebar, re-verifies eligibility itself before dropping anything (defense in depth, not just trusting the caller), and the agent has no delete tool at all - confirmed live: asked to clean up scratch tables, it correctly reported candidates and pointed to the sidebar rather than claiming it could act. Last-use tracking (as opposed to creation-time) remains a deliberate, noted simplification - Postgres doesn't track object access time without extra instrumentation, and age alone was judged sufficient for this workspace's actual size. Same-day follow-up, confirmed with the user directly: the age gate stops the agent (or any automated caller) from casually deleting fresh data, but it also blocked a human who's genuinely done with a table right now and knows exactly which one they want gone. Added a second sidebar section, "Delete any scratch table now" - `list_all_scratch_tables()` lists every scratch table with no age filter, and `delete_scratch_tables(..., require_eligibility=False)` skips the age check entirely for that path. `SAFE_NAME` and the protected-name check are never skippable in either mode - only the age gate is conditional.

`RAG data sourcing alternatives (new corpus / self-referential agent)`

CLOSED, HISTORICAL DECISION — not actually open. Both alternatives were considered during Build 3 and explicitly declined in favor of the summary-based approach, which then extended into Build 3.5's docs-RAG corpus as built. Nothing here is pending a future decision.

Living document — appended at the end of each build, not written once and frozen. Reflects hands-on usage through Build 7, complete end to end — a second, cheaper agent now checks every real tool-backed answer against the actual evidence before it's returned, surfaced as a green/red badge in the Streamlit UI and proven (via a dedicated eval harness) to actually catch a bad answer, not just rubber-stamp everything. Not exhaustive documentation for any of these tools, just what's actually been typed in this project so far.